FlyWithLua Dear ImGui Contrib UX & Extensions Guide

FlyWithLua 2.8.20+ -- ImgWindow v2.3.0+ -- Dear ImGui 1.92.8

Comprehensive Reference for Markdown, TimedTooltip, and Modern Interactive Widgets

October 25, 2026
Author: Steven L. Goldberg (`@slgoldberg`) with help from Google Gemini
Target Release: FlyWithLua 2.8.20+ (Built against Dear ImGui 1.92.8 & ImgWindow v2.3.0)

---

1. Overview & Architecture

FlyWithLua integrates the modern `ImgWindow v2 contrib/` extension ecosystem directly into the Lua scripting environment. This brings rich typography, intelligent stationary tooltips, smooth animated toggles, reset-enabled sliders, and mouse-wheel chasers to X-Plane floating windows.

All extension bindings are implemented natively in C++ (`src/FloatingWindows/FWLContribBindings.cpp`), exposing zero-overhead wrappers that bridge Lua scripts to `ImgWindow`'s header-only extension library.

Key Capabilities

* **Semantic Markdown Typography:** Render rich Markdown text with headings, bold, italics, monospace code blocks, unordered and numbered lists, and clickable hyperlinks.
* **Visual Pills, Badges & Custom Colors:** Inline `<pill>`, `<badge>`, `<kbd>`, and `<color #rrggbb>` formatting for cockpit annunciators, system status, and flight telemetry.
* **Intelligent Stationary Tooltips (`TimedTooltip`):** Stationary hover tooltips that do not jitter or track the mouse, feature reading-adaptive dwell timeouts, enforce layout style isolation, and fully support embedded Markdown.
* **Tactile UX Widgets:** Smooth animated `ToggleButton`, segmented `ToggleButtons`, `SliderWithReset` (with reset icon button), `PercentSlider`, mouse-wheel scroll chasers, and hyperlink widgets.
* **Universal C++ & PascalCase Alias Parity:** Strict parity with C++ Dear ImGui casing (`imgui.Markdown.Text`, `imgui.TimedTooltip.Text`, `imgui.ToggleButton`), accompanied by short aliases (`imgui.MD.Text`, `MD.Text`, `TimedTooltip.Text`) and case-tolerant fallbacks.
* **Variadic String Formatting:** Pass `printf`-style format specifiers (`%s`, `%d`, `%.2f`) directly into any text or tooltip function without needing manual calls to `string.format()`.

---

2. Markdown Extension (`imgui.Markdown` / `MD`)

The Markdown extension parses standard Markdown syntax and renders it as native Dear ImGui draw commands within the current window.

Semantic Font Auto-Wiring

The Markdown system automatically hooks into FlyWithLua's **Named Font Registry**:

Markdown Element	Target Font Name	Fallback Font	Visual Style
Heading 1 (`#`)	`Roboto-Medium-20`	`Roboto-Medium`, Default	20px Bold / Underlined
Heading 2 (`##`)	`Roboto-Medium-16`	`Roboto-Medium`, Default	16px Bold / Underlined
Heading 3 (`###`)	`Roboto-Medium-16`	`Roboto-Medium`, Default	16px Bold
Bold (`**bold**`)	`Roboto-Medium`	`Roboto-Regular`, Default	Medium / Semi-Bold weight
Italics (`*italic*`)	`Roboto-Italic-16`	`Roboto-Italic`, `Roboto-Light`	True Italic / Slanted weight
Monospace / Code (`code`)	`RobotoMono-Regular-16`	`RobotoMono-Regular`, `ProggyForever`	Scalable Monospace font

(Note: True italic and scalable monospace typographies are powered by `Roboto-Italic.ttf` and `RobotoMono-Regular.ttf`, bundled directly in `FlyWithLua/Custom_Fonts/`. If not present, the engine gracefully falls back to `Roboto-Light` or `ProggyForever`).

Markdown Functions

`imgui.Markdown.Text(format_string, ...)`

Renders formatted Markdown text. Supports variadic arguments.

```
imgui.Markdown.Text("# Pre-Flight Inspection")
imgui.Markdown.Text("Verify engine oil pressure: **%.1f PSI** (Nominal: %d-%d PSI)", oil_psi, 45, 60)
```

`imgui.Markdown.TextWrapped(format_string, ...)`

Renders formatted Markdown text with automatic line wrapping to the current window boundary.

```
imgui.Markdown.TextWrapped("This is a long flight briefing paragraph that wraps smoothly across the window...")
```

`imgui.Markdown.TextUnformatted(string)`

Renders Markdown text directly without variadic format parsing (safe for strings containing arbitrary `%` characters).

```
imgui.Markdown.TextUnformatted("Direct text containing 100% pure data.")
```

`imgui.Markdown.ButtonMD(label, [tooltip], [wrap_width])`

Renders a button whose label is parsed with full Markdown support. Returns `true` when clicked.

```
if imgui.Markdown.ButtonMD("Confirm **Auto-Land**##btn", "Engage dual-channel CAT III approach", 300) then
    engage_autoland()
end
```

`width, height = imgui.Markdown.CalcTextSize(text, [wrap_width])`

Calculates the pixel dimensions required to render a given Markdown string before drawing.

```
local w, h = imgui.Markdown.CalcTextSize("### Heading Example")
```

---

3. Inline Styling: Pills, Badges, Keys & Colors

FlyWithLua supports inline XML-style tags embedded directly inside Markdown text:

1. Status Pills (`<pill color>TEXT</pill>`)

Pills render rounded, solid-colored badges with contrasting text -- ideal for system annunciators and warning flags.

Supported color keywords: `green`, `red`, `blue`, `yellow`, `orange`, `purple`, `gray`, `error`, `warning`, `caution`, `info`, `success`.

```
imgui.MD.Text("Hydraulics: <pill green>SYSTEM NORMAL</pill> | Fuel Tank A: <pill warning>LOW LEVEL</pill>")
```

2. Version & Topic Badges (`<badge color>TEXT</badge>`)

Badges render slightly more subtle categorized chips for metadata, build tags, and feature markers.

```
imgui.MD.Text("FlyWithLua <badge blue>v2.8.18</badge> running on <badge purple>Metal / Vulkan</badge>")
```

3. Keyboard Shortcut Tags (`<kbd>Key</kbd>`)

Renders a bordered, keycap-styled box for visual keyboard shortcuts.

```
imgui.MD.Text("Press <kbd>Shift</kbd> + <kbd>Ctrl</kbd> + <kbd>R</kbd> to reload Lua scripts.")
```

4. Custom Hex Colors (`<color #rrggbb>Text</color>`)

Colors arbitrary text spans using standard 6-digit hex color codes.

```
imgui.MD.Text("Target speed: <color #00bc8c>250 knots</color> below <color #f39c12>10,000 ft</color>.")
```

5. Multi-Line Fenced Code Blocks (```lang)

Standard CommonMark triple backticks (or tildes ~~~) render multi-line code blocks in shaded charcoal containers with rounded corners and an automatic language indicator chip in the upper corner:

```
```lua
-- Autopilot vertical speed calculation
local target_vvi = (target_alt - current_alt) * 0.1
if target_vvi > 2000 then target_vvi = 2000 end
set("sim/cockpit2/autopilot/vvi_dial_fpm", target_vvi)
```
```

6. Clickable Hyperlinks

Hyperlinks are currently written using curly-brace syntax: `{Label}(URL)` (curly braces are temporarily used instead of square brackets in this release to prevent link-parsing collisions with technical cockpit annotations and array indices). When clicked, FlyWithLua automatically invokes the native operating system browser (via `open` on macOS, `ShellExecute` on Windows, or `xdg-open` on Linux).

```
imgui.MD.Text("For aircraft manuals, see the {FlyWithLua Docs}(https://github.com/sparker256/FlyWithLua).")
```

---

4. TimedTooltip Extension (`imgui.TimedTooltip` / `TimedTooltip`)

The standard Dear ImGui tooltip (`imgui.SetTooltip`) suffers from several UX limitations in desktop flight simulators:

* **Cursor tracking / Jitter:** The tooltip position moves with every sub-pixel mouse movement, distracting the pilot.
* **Zero Reading Intelligence:** Disappears instantly when the mouse nudges off the target, regardless of how much text remains to read.
* **Style Contamination:** Inherits whatever window padding, spacing, or color overrides happen to be active on the widget stack.

`TimedTooltip` completely resolves these issues through a dedicated tooltip controller:

1. **Stationary Anchor:** Anchors at the position where hover was initiated and remains rock-solid until dismissed.
2. **Reading-Adaptive Timeout:** Dwell duration scales dynamically based on the word count of the tooltip content.
3. **Style Isolation:** Rendered in an independent style context with clean standard padding and borders.
4. **Rich Markdown Formatting:** Supports full headings, bold, italics, pills, badges, and colors via `TimedTooltip.MD.Text`.

TimedTooltip Functions

`imgui.TimedTooltip.Text(format_string, ...)`

Shows a stationary plain text tooltip when the preceding widget is hovered.

```
imgui.Button("AP Master##btn")
imgui.TimedTooltip.Text("Flight Director and Autopilot Master Engagement Switch")
```

`imgui.TimedTooltip.MD.Text(format_string, ...)`

Shows a stationary tooltip rendered with full Markdown and pill styling.

```
imgui.Button("Pitot Heat##btn")
imgui.TimedTooltip.MD.Text("### Pitot Tube Heating System\n\n" ..
                           "Status: <pill green>ON</pill>\n\n" ..
                           "* Prevents ice accumulation in pitot static probes.\n" ..
                           "* <color #f39c12>Caution: Ground operation limited to 5 minutes.</color>")
```

Dwell & Dimension Controls

Configure tooltip behavior globally or per session:

* `imgui.TimedTooltip.SetHoverDelay(cycles)` -- Number of frames cursor must hover before tooltip appears.
* `imgui.TimedTooltip.SetMaxDisplay(cycles)` -- Base display timeout in frames.
* `imgui.TimedTooltip.SetMaxWidth(pixels)` -- Maximum wrap width in pixels (defaults to 400px).
* `imgui.TimedTooltip.SetMinWidth(pixels)` -- Minimum width in pixels.

* `imgui.TimedTooltip.Clear()` -- Immediately dismisses any active tooltip.

5. Modern Contrib Widgets

The `contrib/widgets/` family adds tactile, specialized controls for simulation settings and telemetry adjustment.

`changed, new_val = imgui.ToggleButton(label, bool_val)`

An animated iOS-style sliding pill toggle switch with smooth tactile color transitions.

```
local changed, new_state = imgui.ToggleButton("Cabin Lights", cabin_lights_on)
if changed then
    cabin_lights_on = new_state
end
```

`changed, new_state = imgui.ToggleButtons(label, int_state, off_label, on_label)`

A segmented multi-state toggle control.

```
local changed, new_mode = imgui.ToggleButtons("##RecogLights", light_mode, "OFF", "STROBE")
if changed then
    light_mode = new_mode
end
```

`changed, new_val = imgui.SliderWithReset(label, val, min, max, default_val)`

A horizontal slider paired with an adjacent reset button ([reset]) that instantly returns the slider to its calibrated default value.

```
local changed, new_trim = imgui.SliderWithReset("Aileron Trim", aileron_trim, -1.0, 1.0, 0.0)
if changed then
    aileron_trim = new_trim
end
```

`changed, new_val = imgui.PercentSlider(label, val)`

A slider designed for normalized floating point values `[0.0 .. 1.0]`, automatically displayed and formatted as an integer percentage (`0% .. 100%`).

```
local changed, new_flaps = imgui.PercentSlider("Flaps Deflection", flaps_deploy)
if changed then
    flaps_deploy = new_flaps
end
```

`changed, new_val = imgui.PercentSliderWithReset(label, val, default_val)`

Combines the normalized percent slider with an automatic reset button.

```
local changed, new_mix = imgui.PercentSliderWithReset("Mixture", fuel_mixture, 1.0)
if changed then
    fuel_mixture = new_mix
end
```

`changed, new_val = imgui.AdjustItemOnMouseWheel(val, min, max, [step])`

Detects mouse wheel scrolling when hovering over the preceding item (e.g. a button or custom area) and increments or decrements the floating-point value.

```
imgui.Button(string.format("NAV 1: %.2f MHz", nav1_freq), 220, 30)
local changed, new_freq = imgui.AdjustItemOnMouseWheel(nav1_freq, 108.00, 117.95, 0.05)
if changed then
    nav1_freq = new_freq
end
```

`changed, new_idx = imgui.CycleItemOnMouseWheel(idx, count, [wrap])`

Cycles an integer index when the mouse wheel is scrolled over the preceding widget. Supports standard 1-based Lua indexing.

```
local gear_modes = { "UP", "OFF", "DOWN" }
imgui.Button(string.format("Gear Lever: %s", gear_modes[gear_idx]), 200, 30)
local changed, new_idx = imgui.CycleItemOnMouseWheel(gear_idx, #gear_modes, true)
if changed then
    gear_idx = new_idx
end
```

`clicked = imgui.ButtonLink(label, [url])` and `imgui.TextLink(label, [url])`

Renders a button or text item styled as a hyperlink. If `url` is supplied, clicking automatically opens the URL in the system browser.

```
imgui.TextLink("Check METAR Weather Reports", "https://aviationweather.gov/metar")
imgui.ButtonLink("FlyWithLua Forum Support", "https://forums.x-plane.org")
```

`imgui.Spinner(label, [radius], [thickness], [color], [speed])`

Renders an organic, sinusoidal expanding-and-contracting activity arc for loading or asynchronous tasks. Procedurally rendered via `ImDrawList` antialiased paths with zero textures and zero font dependencies.

* `label`: Unique ID or optional text label rendered beside the spinner.
* `radius`: Outer radius in pixels (defaults to `0.55 * GetFontSize()`).
* `thickness`: Arc stroke width (defaults to `radius * 0.22`, min 2px).
* `color`: RGBA color table `{r, g, b, [a]}` or 32-bit packed color integer (defaults to theme checkmark / button active).
* `speed`: Animation speed multiplier (defaults to `1.0`).

```
-- Standard themed spinner
imgui.Spinner("Loading Flight Plan...")

-- Custom styled aviation-blue spinner at 1.2x speed
imgui.Spinner("FMS Ingestion##fms", 16, 3.0, {0.2, 0.7, 1.0, 1.0}, 1.2)
```

`imgui.RadialTimer(label, fraction, [radius], [thickness], [color], [bg_color], [clockwise])`

Renders a circular countdown dial representing progress or elapsed time (`0.0 .. 1.0`).

* Sweeps from the 12 o'clock position (`-90°`).
* Supports custom arc color and background ring track color.
* `clockwise`: `true` for clockwise sweep (default), `false` for counter-clockwise.

```
-- 65% complete fuel burn cycle dial
imgui.RadialTimer("Fuel Burn Dial", 0.65, 18, 3.5, {0.2, 0.85, 0.3, 1.0})
```

`imgui.CountdownTimer(label, current_sec, max_sec, [radius], [thickness], [color], [bg_color], [clockwise])`

Convenience helper wrapping `RadialTimer` with remaining seconds and total max seconds.

```
-- 42 seconds remaining of a 60 second approach timer
imgui.CountdownTimer("Approach Timer (42s / 60s)", 42.0, 60.0, 20, 4.0, {1.0, 0.65, 0.0, 1.0})
```

`clicked = imgui.ResetButton(str_id, [is_modified], [draw_frame])`

A standalone DPI-scaled vector undo/reset button (`[reset]`) matching standard ImgWindow widget sizing.

* `is_modified`: `true` renders the glyph in standard white/active styling. `false` (baseline) renders it dimmed/disabled.
* `draw_frame`: Whether to draw a button bounding frame or leave it flat/transparent.

```
local reset_clicked = imgui.ResetButton("##reset_zoom", current_zoom ~= 100, false)
if reset_clicked then
    current_zoom = 100
end
```

`clicked, is_copied = imgui.CopyButton(str_id, text_to_copy)`

Renders a 1-click clipboard copy button with automatic transient visual feedback (changing label to `Copied!` in green for 1.5 seconds) and built-in `TimedTooltip` integration.

* Also aliased as `imgui.CopyBufferButton`.

```
imgui.CopyButton("##copy_notes", editor_text)
```

`clicked, is_copied = imgui.CopyLastItemButton(str_id, [prefix], [suffix])`

A non-intrusive chaser copy button that attaches to the preceding widget on the same line (such as an `InputText` or label) and copies its content with optional prefix and suffix formatting.

* Also aliased as `imgui.CopyTextFieldButton`.

```
imgui.InputText("##airport_code", icao_buf, 8)
imgui.SameLine()
imgui.CopyLastItemButton("##copy_icao")
```

`start_idx, end_idx, cur_pos = imgui.GetInputTextSelection()`

Inspects the active Dear ImGui text input or multiline buffer.

* Returns 0-based character indices `[start_idx, end_idx)` and current cursor position `cur_pos`.
* Returns `-1, -1, -1` when no input text item is active or no selection is made.

`imgui.SetInputTextSelection(start_idx, end_idx)`

Programmatically sets or restores the active selection range in the active text input box and automatically reactivates the input widget so keyboard focus is retained.

```
-- Wrap selected text in bold asterisks
local s, e = imgui.GetInputTextSelection()
if s >= 0 and e > s then
    editor_text = editor_text:sub(1, s) .. "**" .. editor_text:sub(s + 1, e) .. "**" .. editor_text:sub(e + 1)
    imgui.SetInputTextSelection(s + 2, e + 2)
end
```

`delta = imgui.GetMouseWheel([consume])` and `imgui.GetMouseWheelEither([consume])`

Low-level mouse wheel query primitives.

* `imgui.GetMouseWheel(consume)`: Returns vertical scroll wheel clicks.
* `imgui.GetMouseWheelEither(consume)`: Returns vertical scroll clicks, falling back to horizontal scroll clicks (ideal for Mac trackpads and horizontal mice).
* When `consume` is `true`, immediately consumes the event by zeroing `io.MouseWheel` so underlying parent windows do not scroll inadvertently.

Standalone Procedural Badges, Pills & Keycaps

While badges and pills are frequently used inline inside Markdown strings, FlyWithLua also exposes them as standalone procedural widgets:

* `imgui.Badge(text, [variant])` -- Subtle categorization chip.
* `imgui.Pill(text, [variant])` -- Solid-colored rounded pill.
* `imgui.Kbd(text)` -- Framed keyboard keycap.

Variants: `"default"`, `"primary"`, `"success"` (`"green"`), `"warning"` (`"yellow"`), `"danger"` (`"red"`), `"info"` (`"blue"`).

```
imgui.Badge("METAR OK", "success")
imgui.SameLine()
imgui.Pill("AUTOPILOT ENGAGED", "primary")
imgui.SameLine()
imgui.Kbd("Ctrl+Shift+P")
```

---

6. Dynamic ImGui Mouse Cursors

FlyWithLua now natively enables **dynamic software ImGui mouse cursors by default** inside floating windows:

* **Interactive Hand (`ImGuiMouseCursor_Hand`):** Shows automatically when hovering over links, buttons, reset glyphs, and toggle switches.
* **Text I-Beam (`ImGuiMouseCursor_TextInput`):** Appears when hovering over editable text inputs.

* **Resize Cursors (`ImGuiMouseCursor_ResizeEW`, `ResizeNS`, etc.):** Seamlessly transition when dragging columns, tables, or frames.
* **Edge Resize Passthrough:** As the mouse approaches the outer boundary of an in-sim window, ImGui automatically releases cursor control to X-Plane so native hardware resize handles function without hitching.

Disabling or Customizing Cursors

If a user or script author prefers standard OS hardware cursor rendering, dynamic cursors can be toggled at multiple levels:

1. **Global Preference (`fwl_prefs.ini`):**

```
; Set to 0 to disable dynamic ImGui cursors globally
ImguiMouseCursors = 0
```

2. **Per-Window Lua Control:**

```
-- Disable custom cursors on this window:
float_wnd_set_imgui_cursor_handling(wnd, false)

-- Query cursor state:
local is_active = float_wnd_get_imgui_cursor_handling(wnd)
```

3. **ImGui Namespace Control:**

```
imgui.SetCursorHandling(false) -- Disable globally
imgui.SetCursorHandling(wnd, false) -- Disable for specific window
```

---

7. Universal C++ & Lua Alias Matrix

FlyWithLua provides comprehensive alias coverage matching both standard Dear ImGui PascalCase conventions and C++ namespace aliases:

Category	Primary API	Short Alias	Global Scope	Suffix Style
Markdown Text	`imgui.Markdown.Text`	`imgui.MD.Text`	`Markdown.Text` / `MD.Text`	`imgui.TextMD`
Markdown Wrapped	`imgui.Markdown.TextWrapped`	`imgui.MD.TextWrapped`	`Markdown.TextWrapped`	`imgui.TextWrappedMD`
Markdown Direct	`imgui.Markdown.TextUnformatted`	`imgui.MD.TextUnformatted`	`Markdown.TextUnformatted`	`imgui.TextUnformattedMD`
Markdown Button	`imgui.Markdown.ButtonMD`	`imgui.MD.ButtonMD`	`Markdown.ButtonMD`	`imgui.ButtonMD`
TimedTooltip Plain	`imgui.TimedTooltip.Text`	`imgui.TimedTooltip.text`	`TimedTooltip.Text`	--
TimedTooltip MD	`imgui.TimedTooltip.MD.Text`	`imgui.TimedTooltip.TextMD`	`TimedTooltip.MD.Text`	`TimedTooltip.TextMD`
Toggle Button	`imgui.ToggleButton`	`imgui.toggle_button`	--	--
Segmented Toggles	`imgui.ToggleButtons`	`imgui.toggle_buttons`	--	--
Slider With Reset	`imgui.SliderWithReset`	`imgui.slider_with_reset`	--	`imgui.ResettableSlider`
Percent Slider	`imgui.PercentSlider`	`imgui.percent_slider`	--	`imgui.SliderPercent`
Percent Slider w/ Reset	`imgui.PercentSliderWithReset`	`imgui.percent_slider_with_reset`	--	`imgui.ResettablePercentSlider`
Reset / Undo Button	`imgui.ResetButton`	`imgui.reset_button`	--	--
Mouse Wheel Adjust	`imgui.AdjustItemOnMouseWheel`	`imgui.adjust_item_on_mouse_wheel`	--	--
Mouse Wheel Cycle	`imgui.CycleItemOnMouseWheel`	`imgui.cycle_item_on_mouse_wheel`	--	--
Core Mouse Wheel	`imgui.GetMouseWheel`	`imgui.get_mouse_wheel`	--	`imgui.GetMouseWheelEither`

Activity Spinner	`imgui.Spinner`	`imgui.spinner`	--	`imgui.ActivitySpinner`
Radial Timer Dial	`imgui.RadialTimer`	`imgui.radial_timer`	--	`imgui.ProgressDial`
Countdown Timer	`imgui.CountdownTimer`	`imgui.countdown_timer`	--	`imgui.CountdownDial`
Copy Button	`imgui.CopyButton`	`imgui.copy_button`	--	`imgui.CopyBufferButton`
Copy Last Item	`imgui.CopyLastItemButton`	`imgui.copy_last_item_button`	--	`imgui.CopyTextFieldButton`
Selection Inspect	`imgui.GetInputTextSelection`	`imgui.get_input_text_selection`	--	--
Selection Set	`imgui.SetInputTextSelection`	`imgui.set_input_text_selection`	--	--
Badge Widget	`imgui.Badge`	`imgui.badge`	--	`imgui.Tag`
Pill Widget	`imgui.Pill`	`imgui.pill`	--	--
Keycap Widget	`imgui.Kbd`	`imgui.kbd`	--	`imgui.Key` / `imgui.Keycap`
Hyperlinks	`imgui.ButtonLink`	`imgui.button_link`	--	`imgui.TextLink`
Mouse Cursor State	`imgui.SetCursorHandling`	`imgui.set_cursor_handling`	--	`imgui.EnableMouseCursors`

8. Complete Interactive Script Recipe

Here is a minimal, complete script showing all extensions working together:

```
if not SUPPORTS_FLOATING_WINDOWS then return end

local wnd = float_wnd_create(540, 420, 1, true)
float_wnd_set_title(wnd, "Avionics Contrib Demo")
float_wnd_set_imgui_builder(wnd, "build_avionics_wnd")

local ap_master = true
local altitude_target = 10000

function build_avionics_wnd(wnd, x, y)
    -- Markdown Header with Pill
    MD.Text("# Flight Management System <badge blue>ACTIVE</badge>")
    MD.Text("Current status: <pill green>EN ROUTE</pill> | Baro: **29.92 inHg**")
    imgui.Separator()

    -- Animated Toggle with TimedTooltip
    local changed, new_ap = imgui.ToggleButton("Autopilot Master", ap_master)
    if changed then ap_master = new_ap end
    TimedTooltip.MD.Text("### Autopilot Master Control\n" ..
                         "Toggles coupled lateral and vertical flight directors.\n" ..
                         "<color #00bc8c>Safe to disengage during cruise flight.</color>")

    -- Altitude Target with Mouse Wheel Tuning
    imgui.Spacing()
    imgui.Button(string.format("Target Altitude: %5d ft (Scroll Wheel)##alt", altitude_target), 300, 30)
    local alt_changed, new_alt = imgui.AdjustItemOnMouseWheel(altitude_target, 0, 45000, 500)
    if alt_changed then altitude_target = math.floor(new_alt) end
    TimedTooltip.Text("Scroll mouse wheel over button to adjust target altitude in 500 ft steps.")

    -- Hyperlink to Documentation
    imgui.Spacing()
    imgui.Separator()
    imgui.TextLink("FlyWithLua Online Documentation", "https://github.com/sparker256/FlyWithLua")
end
```

9. Bundled Demonstration Scripts

FlyWithLua includes complete, production-grade example scripts in `FlyWithLua/Scripts (disabled)/` *(to run either script in X-Plane, simply copy or move it from* `Scripts (disabled)/` *into your active* `Scripts/` *folder)*:

1. `imgui_contrib_demo.lua` (Visual Feature Showcase)

A comprehensive interactive sandbox window demonstrating:

* Headings, soft-wrapping, bold, italic, and strikethrough typography.
* Nested multi-line code blocks with syntax highlighting, copy buttons, and CommonMark indent conformance.
* Tactile widgets: animated toggles, segmented toggles, resettable sliders, and percent sliders.
* Procedural spinners, sweep dials, and countdown timers.
* Interactive TimedTooltips with rich embedded Markdown.

2. `imgui_markdown_editor.lua` (In-Cockpit Notes & Scratchpad)

A full-featured, professional in-cockpit notes and briefing pad featuring:

* **Three Workflow Modes:** Fullscreen **Write**, side-by-side **Split**, and distraction-free flight **Preview**.
* **Tactile Canvas Font Zoom:** Range from 50% to 200% with exact 1-point quantum steppers, mouse wheel adjust, and 1-click A/B toggle back to 100% baseline.
* **Dedicated Multi-Note Storage:** Automatic file persistence in `FlyWithLua/Scripts/MarkdownEditor/` with active note file selection, manual Finder file discovery, and instant new note creation.
* **Flight Operations Checklists:** Rapid task insertion and formatting using verify-and-execute `- [ ]` syntax.
* **Aviation Quick Templates:** Pre-configured templates for ATIS copy pads, IFR CRAFT clearances, pre-flight checklists, and flight diversion briefings.
* **About Dialog:** Live Markdown-rendered documentation and credits.